

El Conversor de Moneda Mágica

El Banco Mágico de Gringotts necesita digitalizar su ventanilla de cambio de divisas para los estudiantes de Hogwarts que vienen del mundo *Muggle*.

Tu objetivo es desarrollar una aplicación web basada en el patrón de arquitectura **MVC (Modelo-Vista-Controlador)** utilizando **Java Servlets**. La aplicación debe permitir a un usuario introducir una cantidad en **Euros (€)** desde un formulario web y calcular su equivalente en **Galeones de Oro** (la moneda oficial del mundo mágico).

Reglas de Negocio (Conversión)

- La tasa de cambio oficial es: **1 Galeón = 5.00 €**.
- El banco no acepta transacciones con valores negativos. Si el usuario introduce una cantidad menor que cero, el sistema debe denegar la operación y reportar un error.

Requisitos Técnicos y Arquitectura

Para mantener un código limpio, escalable y profesional, la aplicación debe estructurarse obligatoriamente bajo el patrón **MVC** y utilizar un objeto **DTO** para el transporte de datos, imitando la estructura del ejercicio de Login.

Debes organizar tus clases dentro del proyecto `cambioDivisa` respetando el siguiente árbol de paquetes:

- `com.centrosanluis.cambioDivisa.dto`
- `com.centrosanluis.cambioDivisa.models`
- `com.centrosanluis.cambioDivisa.controllers`

Instrucciones Paso a Paso

El Objeto de Transferencia de Datos (**ConversionDTO**)

Crea la clase `ConversionDTO` dentro del paquete `.dto`. Este objeto servirá de "paquete" para mover los datos a través de las capas de la aplicación.

- **Atributos privados:** `euros` (double), `galeones` (double) y `mensajeError` (String).
- **Constructor:** Debe recibir únicamente los `euros` introducidos por el usuario.
- **Métodos:** Genera todos sus *Getters* y *Setters*.

La Lógica de Negocio (**ConversorModel**)

Crea la clase `ConversorModel` dentro del paquete `.models`. Esta clase representa el **Modelo** y no debe tener ninguna relación con el entorno web (ni `request`, ni `response`).

- **Constante:** Define una constante privada con la tasa de cambio (\$5.00\$).
- **Método:** Crea el método `public ConversionDTO calcularGaleones(ConversionDTO conversion)`.
 - Si los euros son negativos, debe rellenar el atributo `mensajeError` del DTO con un aviso y poner los galeones a 0.

- Si el valor es correcto, debe realizar la división correspondiente, guardar el resultado en el atributo `galeones` del DTO y asegurarse de que el `mensajeError` quede como `null`.
- Finalmente, devuelve el DTO modificado.

El Ciclo de Vida del Controlador (**ConversorController**)

Crea un Servlet llamado `ConversorController` dentro del paquete `.controllers` y mápalo en la URL `/conversor`.

- **Atributo:** Declara una variable de clase privada para almacenar tu modelo (`private ConversorModel conversorModel;`).
- **Inicialización Eficiente (`init`):** Sobrescribe el método `init()` del Servlet. **Instancia tu modelo aquí dentro una sola vez** para evitar que Tomcat duplique el objeto en memoria con cada petición.
- **Petición (`doPost`):** Dentro del método `doPost`:
 1. Captura el parámetro `"euros"` del formulario web (recuerda parsearlo a `double`).
 2. Instancia el `ConversionDTO` con esos euros.
 3. Llama al método de tu modelo (el que inicializaste en el `init()`) pasándole el DTO.
 4. **Control de Flujo:** * Si el DTO resultante contiene un error, redirige (`sendRedirect`) a la página de inicio enviando el mensaje por la URL (ej: `index.html?error=...`).
 - Si la conversión es exitosa, redirige a la página de éxito enviando el resultado (ej: `banco.html?galeones=...`).

Las Vistas (HTML)

En tu carpeta `webapp`, asegúrate de tener preparadas las interfaces de usuario:

1. `index.html`: Debe contener un formulario con un `<input type="number" name="euros">` que apunte por método `POST` hacia la acción `/conversor`.
2. `banco.html`: Una pantalla de destino limpia que simule la cámara acorazada de Gringotts indicando que los fondos han sido convertidos.